Encryption in ShortP2P

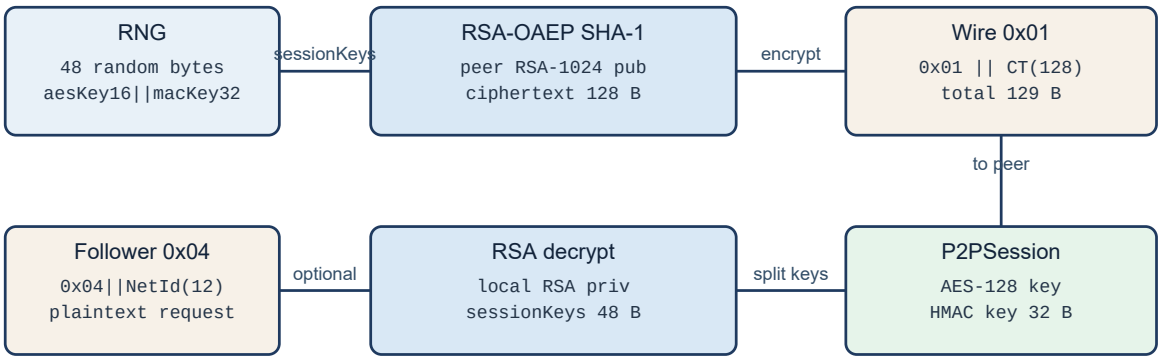
DRAFT - vector sources in docs/encryption/diagrams/*.svg.

Author: Igor Evdokimov
Repository: https://github.com/IskraMessenger/ShortP2P
Created: 2026-11-11
Modified: 2026-11-11
Vector sources: docs/encryption/diagrams/*.svg

Contents

Title page
Contents
1. P2P session establishment (RSA to session keys)
2. P2P symmetric packet (AES-CBC + HMAC)
3. P2P over UDP (data / invite / relay)
4. P2P over BLE
5. P2P over mesh (LoRa / Meshtastic)
6. Client to Server transport (TLS + JWT)
7. Client to Client via Server — payload v2
8. Client to Client via Server — payload v1
9. Client to Server to Bot
10. Message types inside encrypted channels
11. Plaintext frames (no application E2E)
12. Ancillary crypto (non-chat wire)

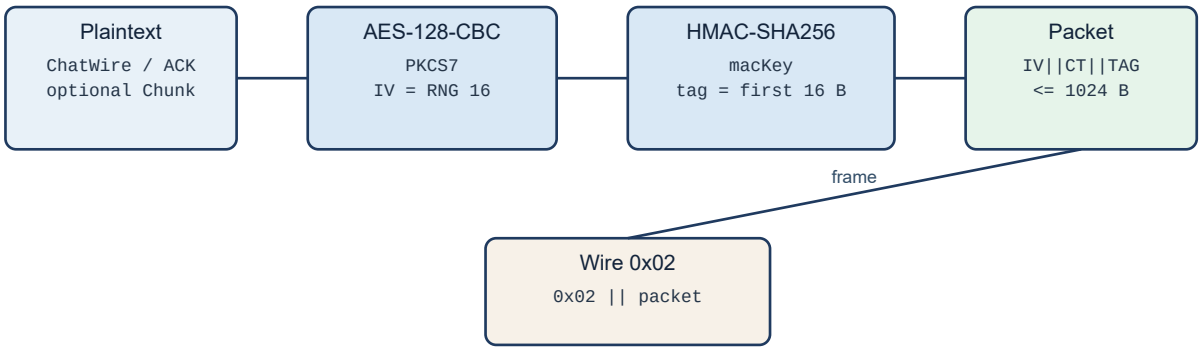
1. P2P session establishment (RSA to session keys)



Leader = smaller NetworkId (12 B). Follower waits for 0x01 after 0x04.

```
Applies to mesh / UDP / BLE before ciphertext messages.
Roles: smaller CompressedNetworkId (12 B) = leader; larger = follower.
Session material: aesKey[16] || macKey[32] (RNG), wrapped with RSA-OAEP(SHA-1).
Wire: leader 0x01||CT(128); follower may send 0x04||NetworkId(12).
Sources: P2PHandshake.cs, P2PCrypto.cs, ChatP2pSession / HandshakeTransceiver.
```

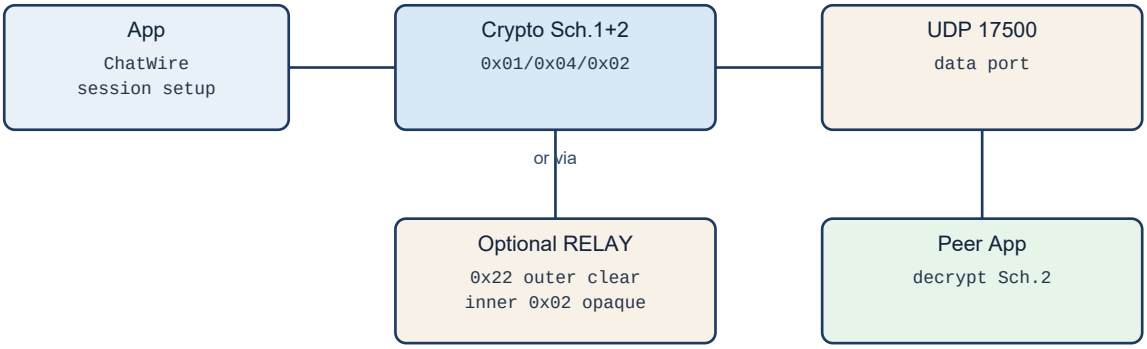
2. P2P symmetric packet (AES-CBC + HMAC)



Decrypt inverse: strip 0x02, verify tag (constant-time), AES-CBC decrypt, unchunk.

```
Shared by UDP / BLE / mesh after session establishment.
AES-128-CBC PKCS7, IV 16 RNG; HMAC-SHA256 truncated to 16; packet IV||CT||TAG <= 1024.
Transport prefix 0x02. Optional ChunkCodec before encrypt.
Sources: P2PSession.cs, ChunkCodec, MessageTransceiver.
```

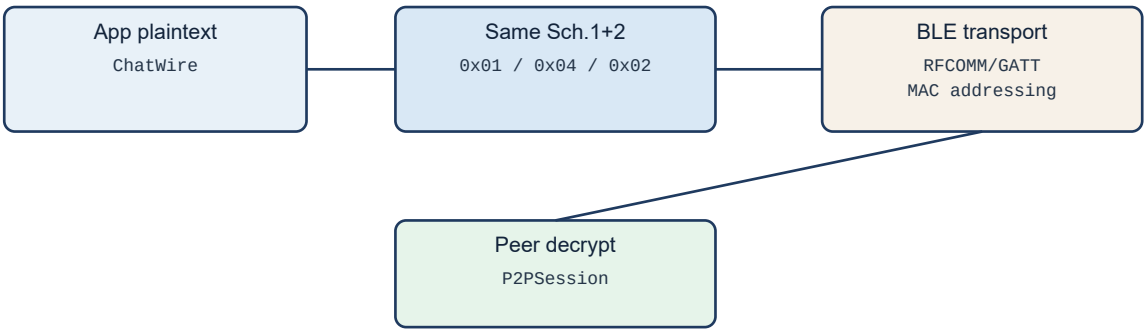
3. P2P over UDP (data / invite / relay)



Invite 17502 / presence 17501 / discovery 17890 stay plaintext.

```
Ports: 17500 data, 17502 invite, 17501 presence, 17890 discovery.
Encrypted chat uses Schemes 1-2 on 17500; RELAY 0x22 may carry opaque 0x02.
Invite/presence/gossip remain plaintext.
```

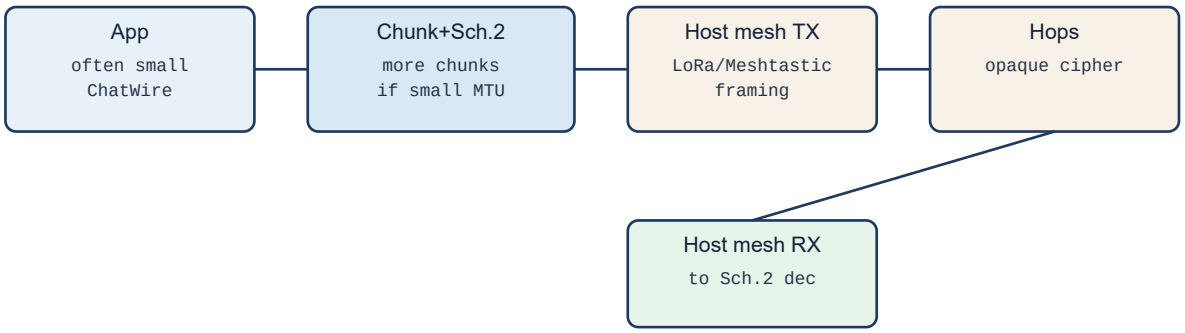
4. P2P over BLE



Identity ads / invite RSA public JSON remain clear on BLE.

```
Same logical crypto packets as UDP (Schemes 1-2) over Bluetooth transport.
Identity advertisements and invite RSA public JSON stay clear.
```

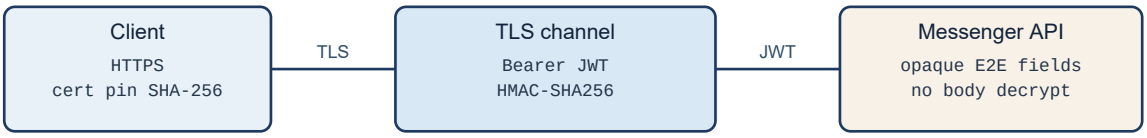
5. P2P over mesh (LoRa / Meshtastic)



Application crypto = Schemes 1-2; mesh host adds transport framing.

```
Application crypto remains Schemes 1-2; host mesh framing is outside ShortP2P.Crypto.
Smaller MTU may increase ChunkCodec fragmentation.
```

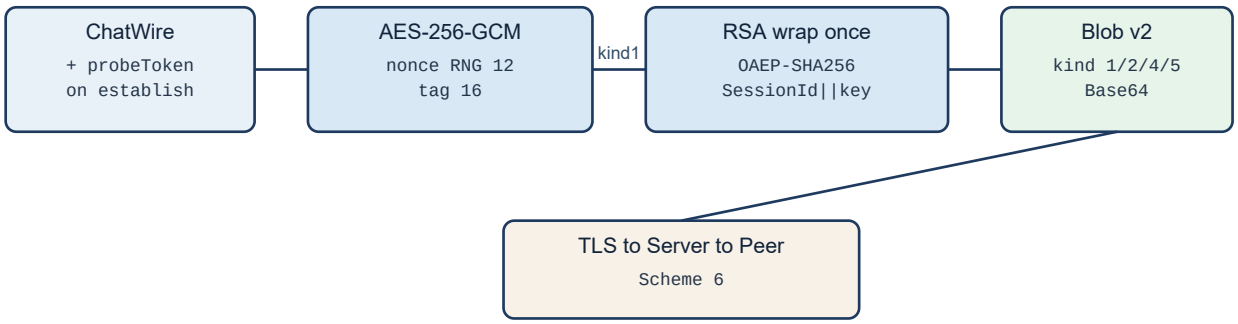
6. Client to Server transport (TLS + JWT)



Protects transport only. Chat secrecy is Schemes 7-9.

```
HTTPS/TLS with optional cert SHA-256 pin; Bearer JWT (HMAC-SHA256).
Server does not decrypt E2E chat bodies (opaque Base64 fields).
```

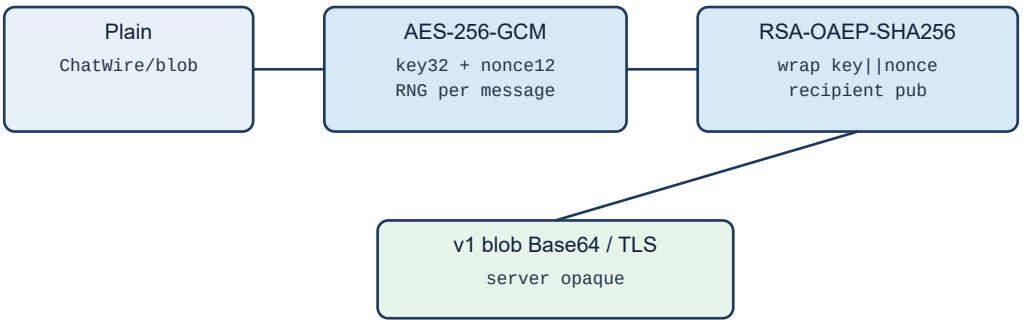
7. Client to Client via Server — payload v2



Later data kind=2 uses SessionId + AES-GCM only (no RSA).

```
Preferred human-peer path: AES-256-GCM session; RSA-OAEP-SHA256 once on establish.
Kinds: 1 establish, 2 data, 4 probe ack, 5 rekey. Then Scheme 6 transport.
Sources: MessengerServerPayloadCodec, ServerCipherSessionStore.
```

8. Client to Client via Server — payload v1



Used for bots and as blob fallback.

```
Per-message hybrid: AES-256-GCM + RSA-OAEP-SHA256 wrap of key||nonce.
Required for bots; blob fallback for humans.
```

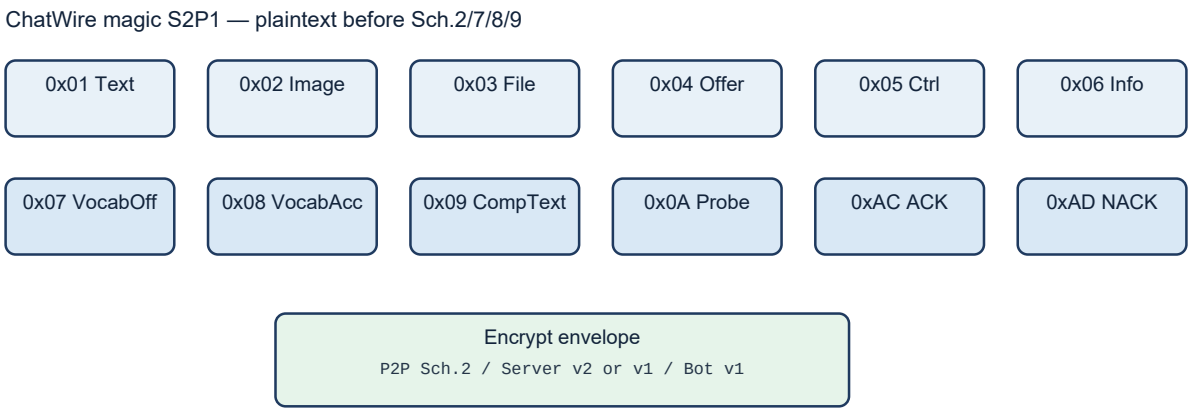
9. Client to Server to Bot



Reply path mirrors: Bot --v1--> Server --v1--> Client.

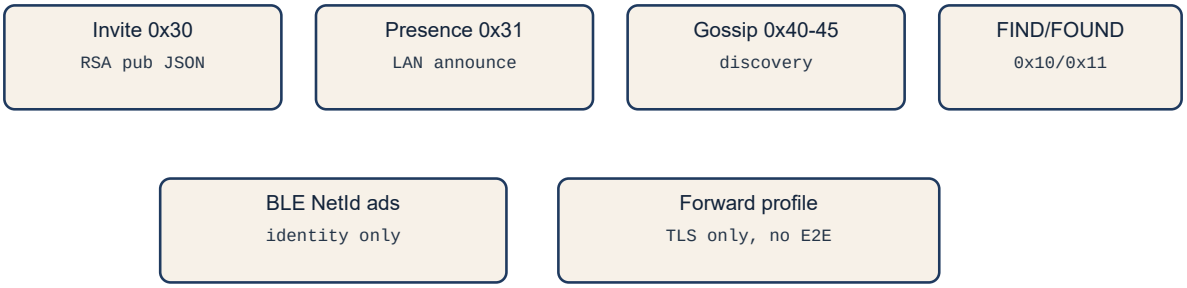
```
E2E uses payload v1 only (no v2 session). Server is opaque relay with TLS+JWT.
Bot auth (cert fingerprint / bot secret) is separate from message E2E.
```

10. Message types inside encrypted channels




```
ChatWire magic S2P1 kinds 0x01-0x0A plus delivery ACK/NACK 0xAC/0xAD.
These are plaintext inputs to Schemes 2 / 7 / 8 / 9.
Source: ChatWireCodec.cs.
```

11. Plaintext frames (no application E2E)



Clear control/identity must not be confused with encrypted ChatWire paths.

```
Invite, presence, gossip, FIND/FOUND, BLE ads, forward profiles (TLS only).
Do not confuse with encrypted ChatWire payloads.
```

12. Ancillary crypto (non-chat wire)



Not used as the chat packet cipher path.

```
SafetyNumber, profile .tlp backup (PBKDF2 + ChaCha20-Poly1305), RSA-1024 identity.
```